SRI KRISHNA
COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution|Anna University Affiliated | NAAC A++ Accredited
Kuniamuthur, Coimbatore – 641008

SRI KRISHNA INSTITUTIONS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

23CSC02 – MACHINE LEARNING TECHNIQUES LABORATORY

RECORD NOTE

Submitted by

Name :

Register No :

Degree & Branch : B.E Computer Science and Engineering (Cyber Security)

Class : II B.E CSE (CS)-A

SRI KRISHNA
COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution|Anna University Affiliated | NAAC A++ Accredited
Kuniamuthur, Coimbatore – 641008

SRI KRISHNA INSTITUTIONS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

23CSC02 – MACHINE LEARNING TECHNIQUES LABORATORY

Practical record submitted by

Name: ……..…….……..… **Register No. :….....……**

Class: II B.E CSE (CS)-A **Degree & Branch: B.E CSE (CS)**

BONAFIDE CERTIFICATE

This is to certify that this is a Bonafide record of work done by

Mr./Ms._____(Register No: _____)

during the academic year 2025 – 2026.

Faculty In-charge **Head of the Department**

Submitted for the University practical examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

LIST OF EXPERIMENTS

EX.NO.	DATE	NAME OF THE EXPERIMENT	PAGE NO.
1.	07.02.2026	Implementation of Regression Models Linear, Polynomial, Ridge & Lasso Regression on real-world data (House Price Prediction)	1
2.	13.02.2026	Decision Tree and Ensemble Learning Implement Decision Tree, Random Forest, and XGBoost for classification tasks (Iris/Titanic dataset)	6
3.	13.02.2026	ISupport Vector Machines (SVM) and K-Nearest Neighbors (KNN) Train and evaluate classifiers on Handwritten Digit Recognition (MNIST)	10
4.	19.02.2026	Artificial Neural Networks (ANN) Build a feedforward neural network for classification using Backpropagation (MNIST dataset)	15
5.	04.03.2026	Convolutional Neural Networks (CNN) Train a CNN model for image classification using CIFAR-10/Fashion-MNIST dataset	19
6.	06.03.2026	Unsupervised Learning Implement K-Means, Hierarchical Clustering, and DBSCAN for customer segmentation	23
7.	10.03.2026	Dimensionality Reduction & Anomaly Detection Apply PCA & t-SNE for feature reduction, Autoencoders for fraud detection.	27
8.	16.03.2026	Model Deployment & Explainable AI Deploy a trained ML model using Flask/FastAPI and analyze model predictions using SHAP/LIME	32

SRI KRISHNA
COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution|Anna University Affiliated | NAAC A++ Accredited
Kuniamuthur, Coimbatore – 641008

SRI KRISHNA INSTITUTIONS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

23CSC02 – MACHINE LEARNING TECHNIQUES LABORATORY

RUBRICS

Criteria	Exp1	Exp2	Exp3	Exp4	Exp5	Exp6	Exp7	Exp8
Aim & Algorithm (20 marks)								
Procedure (30 marks)								
Compilation & Debugging (30 marks)								
Execution & Results (10 marks)								
Documentation & Viva (10 marks)								
Total (100)								

Signature

SRI KRISHNA
COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution|Anna University Affiliated | NAAC A++ Accredited
Kuniamuthur, Coimbatore – 641008

SRI KRISHNA INSTITUTIONS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

23CSC02 – MACHINE LEARNING TECHNIQUES LABORATORY

Record of laboratory work

EVEN SEMESTER - 2025-2026

CONTINUOUS EVALUATION SHEET

REFERENCES RUBRICS TABLE

Criteria	Range of Marks			
	Excellent	Good	Average	Below Average
Aim & Algorithm (20)	18-20	14-17	10-13	0-9
Procedure (30)	27-30	21-26	15-20	0-14
Compilation and Debugging (30)	27-30	21-26	15-20	0-14
Execution and Result (10)	9-10	7-8	5-6	0-4
Documentation and Viva (10)	9-10	7-8	5-6	0-4
Overall Marks	90-100	70-85	50-68	0-45

EX:NO: 1 DATE: 07.02.2026	IMPLEMENTATION OF REGRESSION MODELS: LINEAR, POLYNOMIAL, RIDGE AND LASSO REGRESSION ON REAL WORLD DATA

AIM:

To implement and analyse Linear Regression, Polynomial Regression, Ridge Regression, and Lasso Regression on a real-world House Price Prediction dataset using Python.

ALGORITHM:

STEP-1 Start the visual studio code

STEP-2 Import the libraries

STEP-3 Load the dataset

STEP-4 Identify the input and output columns to train the model

STEP-5 Separate the dataset into training and testing

STEP-6 Finally plot the actual and testing data

STEP-7 END

PROGRAM:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, Ridge, Lasso

from sklearn.preprocessing import PolynomialFeatures, StandardScaler

from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_csv(r"C:\Users\lucky\OneDrive\Desktop\SKCET\MLT Lab Experiments\BostonHousing.csv")

data = data.dropna()
```

```
X = data.drop("medv", axis=1)
y = data["medv"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Linear Regression

```
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)
```

Polynomial Regression

```
poly = PolynomialFeatures(degree=2)
X_poly_train = poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

poly_model = LinearRegression()
poly_model.fit(X_poly_train, y_train)
y_pred_poly = poly_model.predict(X_poly_test)
```

Ridge Regression

```
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train_scaled, y_train)
y_pred_ridge = ridge_model.predict(X_test_scaled)
```

Lasso Regression

```
lasso_model = Lasso(alpha=0.1)
lasso_model.fit(X_train_scaled, y_train)
y_pred_lasso = lasso_model.predict(X_test_scaled)
```

Evaluation Function

```
def evaluate(model_name, y_test, y_pred):
    print(model_name)
    print("MSE:", mean_squared_error(y_test, y_pred))
    print("R2 Score:", r2_score(y_test, y_pred))
    print("----------------------------------")
```

Results

```
evaluate("Linear Regression", y_test, y_pred_linear)
evaluate("Polynomial Regression", y_test, y_pred_poly)
evaluate("Ridge Regression", y_test, y_pred_ridge)
evaluate("Lasso Regression", y_test, y_pred_lasso)
```

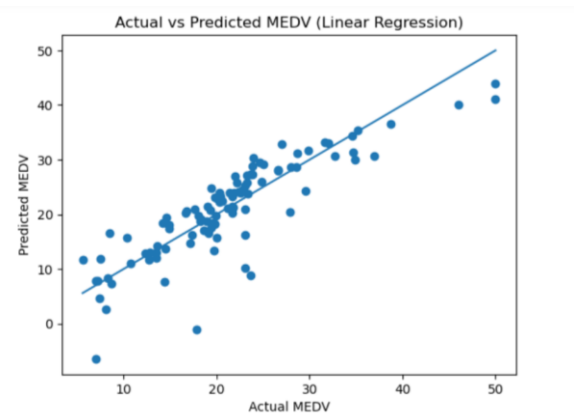
Graphs: Actual vs Predicted Prices

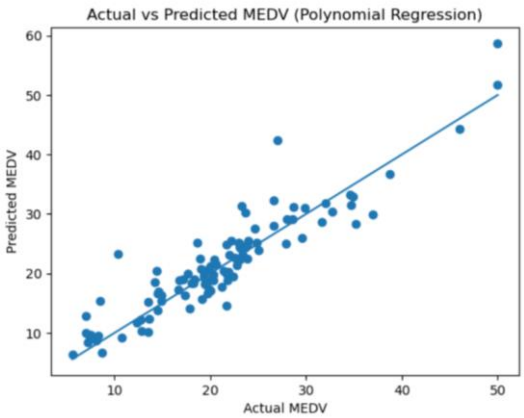
```
models = {
    "Linear": y_pred_linear,
    "Polynomial": y_pred_poly,
    "Ridge": y_pred_ridge,
    "Lasso": y_pred_lasso
}

for name, preds in models.items():
    plt.figure()
```

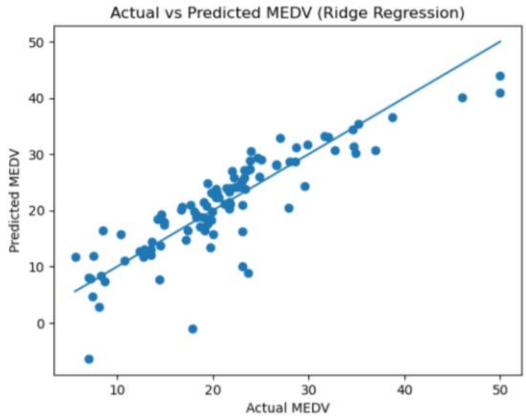
```
plt.scatter(y_test, preds)
plt.plot([y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()])
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title(f"Actual vs Predicted MEDV ({name} Regression)")
plt.show()
```

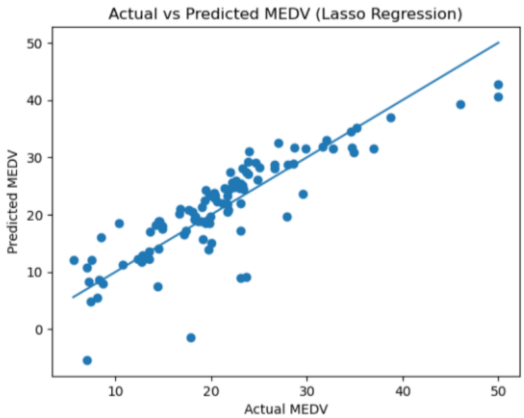
OUTPUT:

```
Linear Regression
MSE: 20.687720473048604
R2 Score: 0.72002776785803
----------------------------------
Polynomial Regression
MSE: 12.45895834096065
R2 Score: 0.8313897182907681
----------------------------------
Ridge Regression
MSE: 20.658202015330758
R2 Score: 0.7204272487243448
----------------------------------
Lasso Regression
MSE: 21.63888776463179
R2 Score: 0.7071553767160526
----------------------------------
```









RESULT:

Thus, Linear, Polynomial, Ridge, and Lasso Regression models were successfully implemented using Python on a real-world house price dataset. Among all models, Polynomial Regression achieved better accuracy, demonstrating its ability to capture non-linear relationships.

EX:NO: 2 DATE: 13.02.2026	DECISION TREE AND ENSEMBLE LEARNING – IMPLEMENT DECISION TREE, RANDOM FOREST, AND XGBOOST FOR CLASSIFICATION TASKS (IRIS DATASET)

AIM:

To implement and compare Decision Tree, Random Forest, and XGBoost classifiers on real-world datasets (Iris and Titanic) and evaluate their performance using appropriate metrics.

ALGORITHM:

STEP 1 - Import Required Libraries

STEP 2 - Load the Dataset

STEP 3 - Split the Dataset

STEP 4 - Train Decision Tree Model

STEP 5 - Train Random Forest Model

STEP 6 - Train XGBoost Model

STEP 7 - Compare Model Performance

STEP 8 - Display Results

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from xgboost import XGBClassifier
```

Load the dataset

```
iris = load_iris()
X = iris.data
y = iris.target
```

Train-Test Spilt

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)
```

Decision Tree Classifier

```
dt = DecisionTreeClassifier(criterion='gini')
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print(confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

Random forest Classifier

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(confusion_matrix(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

XGBoost Clasifier

```
xgb = XGBClassifier(eval_metric='mlogloss')
xgb.fit(X_train, y_train)
```

```
y_pred_xgb = xgb.predict(X_test)
print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
print(confusion_matrix(y_test, y_pred_xgb))
print(classification_report(y_test, y_pred_xgb))
```

OUTPUT:

```
Decision Tree Accuracy: 1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```
Random Forest Accuracy: 1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```
XGBoost Accuracy: 1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

RESULT:

Thus, Decision Tree, Random Forest, and XGBoost classifiers were successfully implemented and evaluated.

EX:NO: 3 DATE: 13.02.2026	SUPPORT VECTOR MACHINES (SVM) AND K-NEAREST NEIGHBORS (KNN) – TRAIN AND EVALUATE CLASSIFIERS ON HANDWRITTEN DIGIT RECOGNITION (MNIST)

AIM:

To implement SVM and KNN classifiers for recognizing handwritten digits using the MNIST dataset and to evaluate their performance using standard classification metrics.

ALGORITHM:

STEP 1 - Start

STEP 2 - Import Required Libraries

STEP 3 - Load the Dataset

STEP 4 - Separate Features and Labels

STEP 5 - Data Preprocessing

STEP 6 - Split the Dataset

STEP 7 - Train the SVM Model

STEP 8 - Predict using SVM

STEP 9 - Evaluate SVM Performance

STEP 10 - Train the KNN Model

STEP 11 - Predict using KNN

STEP 12 - Evaluate KNN Performance

STEP 13 - Visualize Predictions

STEP 14 - Stop

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
```

```
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

mnist = fetch_openml('mnist_784', version=1)
X = mnist.data
y = mnist.target.astype(int)
```

Normalize

```
X = X / 255.0
print("Dataset Loaded Successfully")
```

#Dataset Loaded Successfully - output

Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

SVM Model

```
svm = SVC(kernel='rbf', gamma='scale')
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print(confusion_matrix(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

KNN Model

```
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print(confusion_matrix(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

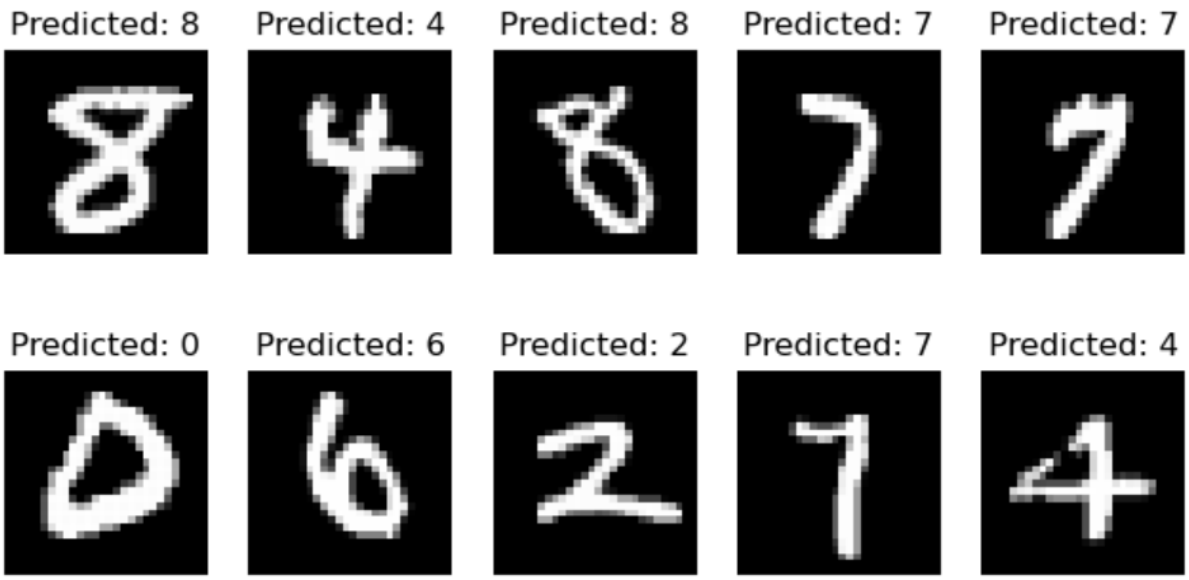
Visualization

```
plt.figure(figsize=(8,4))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_test.iloc[i].values.reshape(28,28), cmap='gray')
    plt.title("Predicted: " + str(y_pred_knn[i]))
    plt.axis('off')
plt.show()
```

OUTPUT:

```
SVM Accuracy: 0.9764285714285714
[[1329    1    3    0    1    2    2    1    4    0]
 [   0 1585    4    3    2    0    0    4    2    0]
 [   3    4 1348    2    3    2    4    8    5    1]
 [   0    2   11 1386    2   11    1    9    7    4]
 [   1    0    2    0 1269    0    3    2    2   16]
 [   0    1    2   15    2 1236    9    1    7    0]
 [   1    0    0    0    4    4 1384    0    3    0]
 [   1    6   12    1    5    1    0 1465    1   11]
 [   2    6    8   12    4    9    6    4 1303    3]
 [   5    8    2    8   14    2    0   11    5 1365]]
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      1343
           1       0.98      0.99      0.99      1600
           2       0.97      0.98      0.97      1380
           3       0.97      0.97      0.97      1433
           4       0.97      0.98      0.98      1295
           5       0.98      0.97      0.97      1273
           6       0.98      0.99      0.99      1396
           7       0.97      0.97      0.97      1503
           8       0.97      0.96      0.97      1357
           9       0.97      0.96      0.97      1420

    accuracy                           0.98     14000
   macro avg       0.98      0.98      0.98     14000
weighted avg       0.98      0.98      0.98     14000
```

```
KNN Accuracy: 0.9700714285714286
[[1336    0    3    0    0    0    2    1    1    0]
 [   0 1592    2    0    1    1    0    3    0    1]
 [   7   17 1323    1    4    1    5   17    3    2]
 [   0    2   12 1384    1    8    2    7    7   10]
 [   3    8    1    0 1251    0    2    3    1   26]
 [   2    5    0   16    2 1232   13    0    1    2]
 [   5    1    0    0    5    6 1379    0    0    0]
 [   1   21    4    0    4    0    0 1458    1   14]
 [   4   13    6   22    2   19    4   10 1267   10]
 [   6    5    2   11   19    0    0   17    1 1359]]
              precision    recall  f1-score   support

           0       0.98      0.99      0.99      1343
           1       0.96      0.99      0.98      1600
           2       0.98      0.96      0.97      1380
           3       0.97      0.97      0.97      1433
           4       0.97      0.97      0.97      1295
           5       0.97      0.97      0.97      1273
           6       0.98      0.99      0.98      1396
           7       0.96      0.97      0.97      1503
           8       0.99      0.93      0.96      1357
           9       0.95      0.96      0.96      1420

    accuracy                           0.97     14000
   macro avg       0.97      0.97      0.97     14000
weighted avg       0.97      0.97      0.97     14000
```



RESULT:

Thus, SVM and KNN classifiers were successfully implemented for handwritten digit recognition using the MNIST dataset. SVM achieved slightly higher accuracy compared to KNN.

EX:NO: 4 DATE: 19.02.2026	ARTIFICIAL NEURAL NETWORKS (ANN) – BUILD A FEEDFORWARD NEURAL NETWORK FOR CLASSIFICATION USING BACKPROPAGATION (MNIST DATASET)

AIM:

To design and implement an Artificial Neural Network (ANN) using TensorFlow to classify handwritten digits from the MNIST Dataset using the backpropagation learning algorithm.

ALGORITHM:

STEP 1: Start the program.

STEP 2: Import the required libraries such as TensorFlow, NumPy, and Matplotlib.

STEP 3: Load the MNIST Dataset using the TensorFlow library.

STEP 4: Split the dataset into training data and testing data.

STEP 5: Normalize the pixel values of the images by dividing by 255 so that values lie between 0 and 1.

STEP 6: Convert the 28 × 28-pixel images into a 784-dimensional vector (flatten the images).

STEP 7: Create a feedforward neural network model using the Sequential model.

STEP 8: Add the first hidden layer with 128 neurons and ReLU activation function.

STEP 9: Add the second hidden layer with 64 neurons and ReLU activation function.

STEP 10: Add the output layer with 10 neurons and SoftMax activation function for digit classification.

STEP 11: Compile the model using Adam optimizer, sparse categorical cross-entropy loss, and accuracy metric.

STEP 12: Train the model using the training dataset with backpropagation for multiple epochs.

STEP 13: Evaluate the trained model using the test dataset to calculate accuracy.

STEP 14: Display the predicted digit and visualize the test image.

STEP 15: Stop the program.

PROGRAM:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
import matplotlib.pyplot as plt
```

Load MNIST dataset

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

Display dataset shapes

```
print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

Normalize the pixel values (0-255 to 0-1)

```
x_train = x_train / 255.0
x_test = x_test / 255.0
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)

model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(784,)))

model.add(Dense(64, activation='relu'))
```

Output Layer (10 classes)

```
model.add(Dense(10, activation='softmax'))
```

Compile the model

```
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

Train the model using Backpropagation

```
history = model.fit(x_train, y_train,
epochs=10,
batch_size=32,
validation_split=0.2)
```

Evaluate the model

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

Make predictions

```
predictions = model.predict(x_test)
```

Display predicted and actual value

```
predicted_label = np.argmax(predictions[0])
actual_label = y_test[0]
print("Predicted Digit:", predicted_label)
print("Actual Digit:", actual_label)
```

Display the image

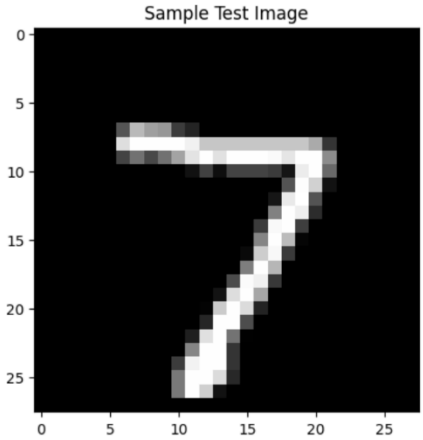
```
plt.imshow(x_test[0].reshape(28,28), cmap='gray')
plt.title("Sample Test Image")
plt.show()
```

OUTPUT:

```
Epoch 1/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9839 - loss: 0.1346 - val_accuracy: 0.9725 - val_loss: 0.1590
Epoch 2/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9891 - loss: 0.0540 - val_accuracy: 0.9714 - val_loss: 0.1583
Epoch 3/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9898 - loss: 0.0522 - val_accuracy: 0.9728 - val_loss: 0.1503
Epoch 4/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9902 - loss: 0.0512 - val_accuracy: 0.9726 - val_loss: 0.1577
Epoch 5/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9909 - loss: 0.0490 - val_accuracy: 0.9734 - val_loss: 0.1462
Epoch 6/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9914 - loss: 0.0423 - val_accuracy: 0.9753 - val_loss: 0.1357
Epoch 7/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9914 - loss: 0.0415 - val_accuracy: 0.9766 - val_loss: 0.1362
Epoch 8/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9930 - loss: 0.0341 - val_accuracy: 0.9681 - val_loss: 0.1840
Epoch 9/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9925 - loss: 0.0385 - val_accuracy: 0.9753 - val_loss: 0.1410
Epoch 10/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9936 - loss: 0.0305 - val_accuracy: 0.9750 - val_loss: 0.1303
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9744 - loss: 0.1377
Test Loss: 0.13766658306121826
Test Accuracy: 0.974399983882904
```

```
Predicted Digit: 7
Actual Digit: 7
```



RESULT:

Thus, a feedforward Artificial Neural Network using Backpropagation was successfully implemented to classify handwritten digits from the MNIST dataset with high accuracy.

EX:NO: 5 DATE: 04.03.2026	CONVOLUTIONAL NEURAL NETWORKS (CNN) – TRAIN A CNN MODEL FOR IMAGE CLASSIFICATION USING CIFAR-10/FASHION-MNIST DATASET

AIM:

To train and evaluate a Convolutional Neural Network (CNN) model for image classification using the Fashion-MNIST dataset in TensorFlow.

ALGORITHM:

STEP 1: Import required libraries such as TensorFlow, Keras, and Matplotlib.

STEP 2: Load the Fashion-MNIST dataset from Keras datasets.

STEP 3: Normalize image pixel values from 0–255 to 0–1.

STEP 4: Reshape images to include the channel dimension (28 × 28 × 1).

STEP 5: Build a CNN model using convolution, pooling, and dense layers.

STEP 6: Compile the model using Adam optimizer and sparse categorical crossentropy loss.

STEP 7: Train the CNN model using the training dataset for several epochs.

STEP 8: Validate the model using the testing dataset.

STEP 9: Evaluate the model performance and compute test accuracy.

STEP 10: Plot training and validation accuracy graphs.

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

Load Fashion MNIST dataset

```
(train_images, train_labels), (test_images, test_labels) =
datasets.fashion_mnist.load_data()
```

Normalize the pixel values

```
train_images = train_images / 255.0
test_images = test_images / 255.0
```

Reshape images for CNN (add channel dimension)

```
train_images = train_images.reshape((60000, 28, 28, 1))
test_images = test_images.reshape((10000, 28, 28, 1))
```

Build CNN model

```
model = models.Sequential()
```

Convolution layer

```
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))
```

Second convolution layer

```
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
```

Third convolution layer

```
model.add(layers.Conv2D(64, (3,3), activation='relu'))
```

Flatten layer

```
model.add(layers.Flatten())
```

Fully connected layer

```
model.add(layers.Dense(64, activation='relu'))
```

Output layer (10 classes)

```
model.add(layers.Dense(10, activation='softmax'))
```

Compile the model

```
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

Train the model

```
history = model.fit(train_images, train_labels, epochs=10,
validation_data=(test_images, test_labels))
```

Evaluate the model

```
test_loss, test_acc = model.evaluate(test_images, test_labels)

print("Test Accuracy:", test_acc)
```

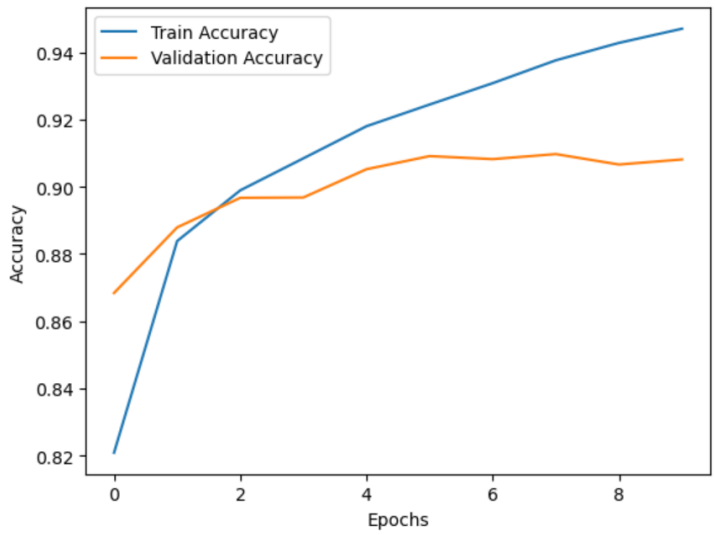
Plot training accuracy

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

OUTPUT:

```
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 30s 15ms/step - accuracy: 0.8209 - loss: 0.4967 - val_accuracy: 0.8684 - val_loss: 0.3673
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 28s 15ms/step - accuracy: 0.8838 - loss: 0.3193 - val_accuracy: 0.8879 - val_loss: 0.3301
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 29s 15ms/step - accuracy: 0.8989 - loss: 0.2756 - val_accuracy: 0.8967 - val_loss: 0.2880
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 30s 16ms/step - accuracy: 0.9084 - loss: 0.2474 - val_accuracy: 0.8968 - val_loss: 0.2781
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 30s 16ms/step - accuracy: 0.9180 - loss: 0.2236 - val_accuracy: 0.9052 - val_loss: 0.2690
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 35s 19ms/step - accuracy: 0.9244 - loss: 0.2036 - val_accuracy: 0.9091 - val_loss: 0.2606
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 41s 22ms/step - accuracy: 0.9308 - loss: 0.1860 - val_accuracy: 0.9082 - val_loss: 0.2698
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 40s 22ms/step - accuracy: 0.9376 - loss: 0.1679 - val_accuracy: 0.9097 - val_loss: 0.2690
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 44s 23ms/step - accuracy: 0.9428 - loss: 0.1551 - val_accuracy: 0.9066 - val_loss: 0.2877
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 43s 23ms/step - accuracy: 0.9470 - loss: 0.1417 - val_accuracy: 0.9081 - val_loss: 0.2899
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9081 - loss: 0.2899
Test Accuracy: 0.9081000089645386
```



RESULT:

The CNN model was successfully trained using the Fashion-MNIST dataset and achieved good classification accuracy on the test dataset.

EX:NO: 6 DATE: 06.03.2026	UNSUPERVISED LEARNING – IMPLEMENT K-MEANS, HIERARCHICAL CLUSTERING, AND DBSCAN FOR CUSTOMER SEGMENTATION

AIM:

To implement Unsupervised Learning algorithms (K-Means, Hierarchical Clustering, and DBSCAN) to perform customer segmentation based on income and spending behaviour.

ALGORITHM:

STEP 1: Import required libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn.

STEP 2: Load the customer dataset using Pandas.

STEP 3: Select relevant features such as Annual Income and Spending Score.

STEP 4: Apply feature scaling using StandardScaler.

STEP 5: Apply K-Means clustering to divide customers into clusters.

STEP 6: Visualize the clusters using a scatter plot.

STEP 7: Apply Hierarchical Clustering and generate a dendrogram.

STEP 8: Use Agglomerative Clustering to form clusters.

STEP 9: Apply DBSCAN clustering to detect density-based clusters.

STEP 10: Visualize and compare clustering results.

PROGRAM:

Import libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
```

Load dataset

```
data = pd.read_csv("Mall_Customers.csv")
```

Select important features

```
X = data[['Annual Income (k$)', 'Spending Score (1-100)']]
```

Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

K-MEANS CLUSTERING

```
kmeans = KMeans(n_clusters=5, random_state=0)
kmeans_labels = kmeans.fit_predict(X_scaled)

plt.scatter(X_scaled[:,0], X_scaled[:,1], c=kmeans_labels, cmap='rainbow')
plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.show()
```

HIERARCHICAL CLUSTERING

```
linked = linkage(X_scaled, method='ward')
plt.figure(figsize=(8,5))
dendrogram(linked)
plt.title("Dendrogram for Hierarchical Clustering")
plt.xlabel("Customers")
plt.ylabel("Distance")
plt.show()

hc = AgglomerativeClustering(n_clusters=5)
hc_labels = hc.fit_predict(X_scaled)

plt.scatter(X_scaled[:,0], X_scaled[:,1], c=hc_labels, cmap='rainbow')
plt.title("Customer Segmentation using Hierarchical Clustering")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.show()
```

DBSCAN CLUSTERING

```
dbscan = DBSCAN(eps=0.5, min_samples=5)
```

```
db_labels = dbscan.fit_predict(X_scaled)

plt.scatter(X_scaled[:,0], X_scaled[:,1], c=db_labels, cmap='plasma')
plt.title("Customer Segmentation using DBSCAN")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.show()
```

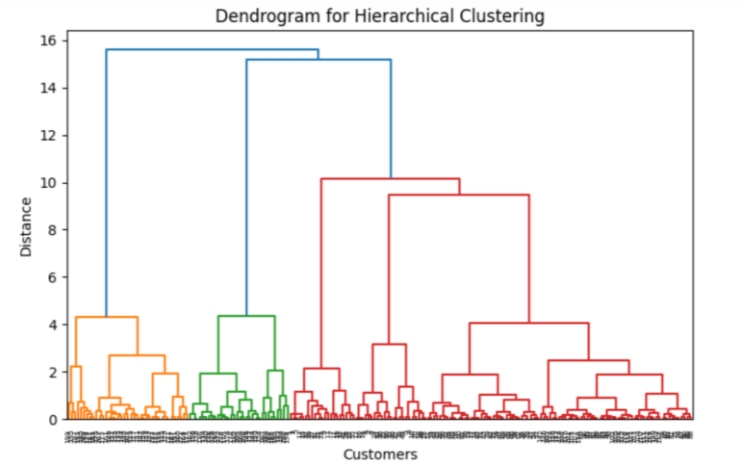
Print cluster labels

```
print("K-Means Clusters:\n", kmeans_labels)
print("Hierarchical Clusters:\n", hc_labels)
print("DBSCAN Clusters:\n", db_labels)
```

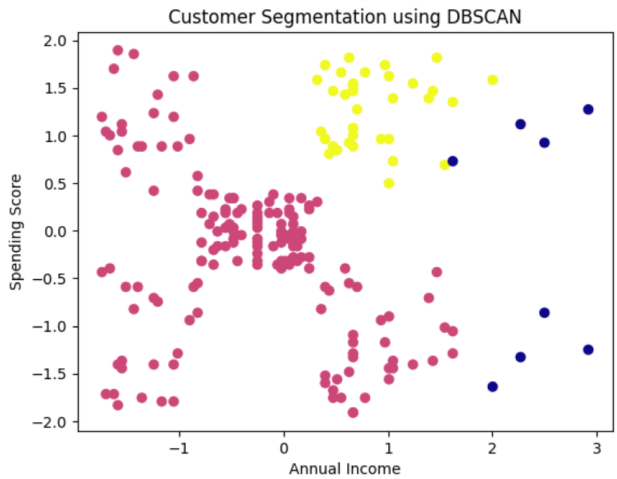
OUTPUT:

K- Means Clustering

Hierarchical Clustering



DBSCAN Clustering




```
K-Means Clusters:
 [3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3
 4 3 4 3 4 3 0 3 4 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 1 2 1 0 1 2 1 2 1 0 1 2 1 2 1 2 1 2 1 0 1 2 1 2 1
 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2
 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1]
Hierarchical Clusters:
 [4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4
 3 4 3 4 3 4 2 4 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 1 2 1 2 1 0 1 0 1 2 1 0 1 0 1 0 1 0 1 2 1 0 1 2 1
 0 1 0 1 0 1 0 1 0 1 0 1 2 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0
 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1]
DBSCAN Clusters:
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1
  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1
  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0  1  0 -1
 -1  1 -1 -1 -1 -1 -1 -1]
```

RESULT:

The K-Means, Hierarchical Clustering, and DBSCAN algorithms were implemented successfully to perform customer segmentation.

EX:NO: 7 DATE: 10.03.2026	DIMENSIONALITY REDUCTION & ANOMALY DETECTION – APPLY PCA & T-SNE FOR FEATURE REDUCTION, AUTOENCODERS FOR FRAUD DETECTION

AIM:

To perform dimensionality reduction using PCA and t-SNE and detect anomalies using Autoencoders for fraud detection.

ALGORITHM:

STEP 1: Start the program.

STEP 2: Import required libraries.

STEP 3: Load the dataset.

STEP 4: Apply PCA to reduce features to 2 dimensions.

STEP 5: Apply t-SNE for dimensionality reduction and visualization.

STEP 6: Normalize the dataset using MinMaxScaler.

STEP 7: Build the Autoencoder model (encoder and decoder layers).

STEP 8: Train the model using the dataset.

STEP 9: Reconstruct the data using the trained model.

STEP 10: Calculate reconstruction error (MSE).

STEP 11: Set threshold and detect anomalies.

STEP 12: Display detected anomalies and stop the program.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.preprocessing import MinMaxScaler
```

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
```

Load Dataset

```
data = load_digits()
X = data.data
y = data.target
```

PCA Dimensionality Reduction

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.figure()
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.title("PCA Result")
plt.show()
```

t-SNE Dimensionality Reduction

```
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)

plt.figure()
plt.scatter(X_tsne[:,0], X_tsne[:,1], c=y)
plt.title("t-SNE Result")
plt.show()
```

Prepare Data for Autoencoder

```
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

input_dim = X_scaled.shape[1]
```

Build Autoencoder Model

```
input_layer = Input(shape=(input_dim,))
```

```
encoder = Dense(32, activation="relu")(input_layer)
encoder = Dense(16, activation="relu")(encoder)

decoder = Dense(32, activation="relu")(encoder)
decoder = Dense(input_dim, activation="sigmoid")(decoder)

autoencoder = Model(inputs=input_layer, outputs=decoder)
autoencoder.compile(optimizer="adam", loss="mse")
```

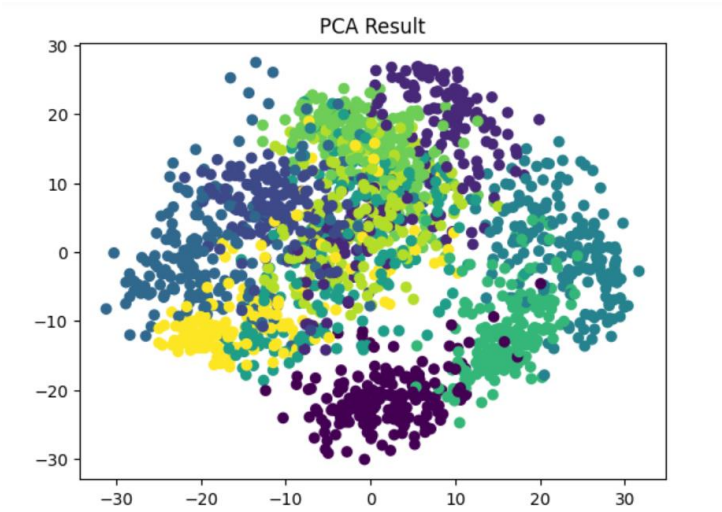
Train Model

```
autoencoder.fit(X_scaled, X_scaled,
epochs=20,
batch_size=32,
shuffle=True)
```

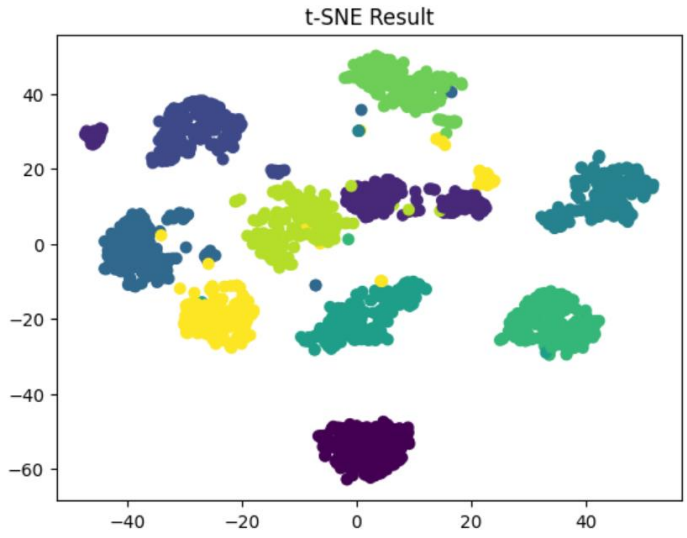
Anomaly Detection

```
reconstructed = autoencoder.predict(X_scaled)
mse = np.mean(np.power(X_scaled - reconstructed, 2), axis=1)
threshold = np.mean(mse) + 2*np.std(mse)
anomalies = mse > threshold

print("Total Anomalies Detected:", np.sum(anomalies))
```

SAMPLE OUTPUT:



t-SNE Dimensionality Reduction




```
Epoch 1/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.1391
Epoch 2/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0740
Epoch 3/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.0625
Epoch 4/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0528
Epoch 5/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.0462
Epoch 6/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 0.0412
Epoch 7/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0369
Epoch 8/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0335
Epoch 9/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.0313
Epoch 10/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0297
Epoch 11/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0282
Epoch 12/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.0270
Epoch 13/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0258
Epoch 14/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0247
Epoch 15/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0237
Epoch 16/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0228
Epoch 17/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.0220
Epoch 18/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.0214
Epoch 19/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.0207
Epoch 20/20
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.0203
```

```
57/57 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
Total Anomalies Detected: 72
```

RESULT:

Dimensionality reduction using PCA and t-SNE successfully reduced high-dimensional data to 2D for visualization.

EX:NO: 8 DATE: 16.03.2026	MODEL DEPLOYMENT & EXPLAINABLE AI – DEPLOY A TRAINED ML MODEL USING FLASK/FASTAPI AND ANALYZE MODEL PREDICTIONS USING SHAP/LIME

AIM:

To deploy a trained Machine Learning model using Flask and analyze the model predictions using SHAP (SHapley Additive Explanations) for interpretability.

ALGORITHM:

STEP 1: Import required libraries such as NumPy, Pandas, Scikit-learn, Flask, and SHAP.

STEP 2: Load the Diabetes dataset from Scikit-learn.

STEP 3: Split the dataset into training and testing sets.

STEP 4: Train a Machine Learning model (Random Forest / Linear Regression).

STEP 5: Create a Flask API to deploy the trained model.

STEP 6: Accept input features through an API request.

STEP 7: Use the trained model to predict output values.

STEP 8: Apply SHAP explainer to analyze feature importance.

STEP 9: Visualize the impact of each feature on the prediction.

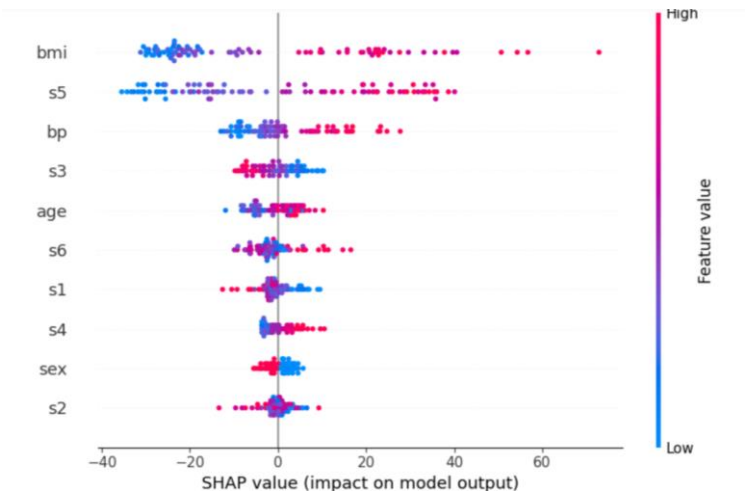
PROGRAM:

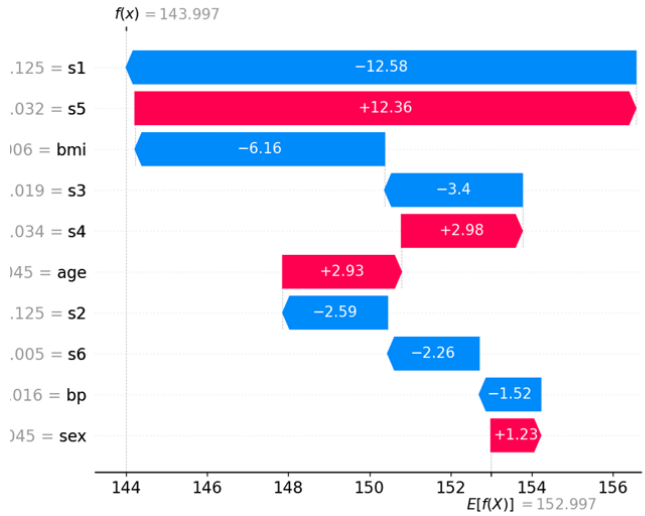
```
import numpy as np
import pandas as pd
import shap
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from flask import Flask, request, jsonify
```

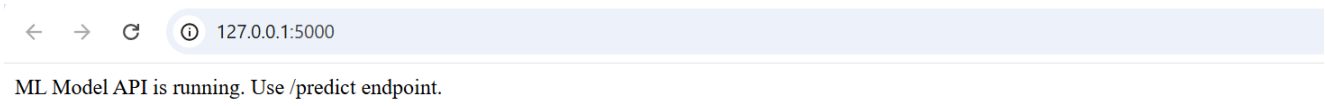
```
data = load_diabetes()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
explainer = shap.Explainer(model, X_train)
shap_values = explainer(X_test, check_additivity=False)
shap.summary_plot(shap_values, X_test)

shap.plots.waterfall(shap_values[0])

shap.initjs()
force_plot = shap.plots.force(shap_values[0])

shap.save_html("shap_force_plot.html", force_plot)
print("SHAP interactive visualization saved as shap_force_plot.html")

# Flask app
app = Flask(__name__)

@app.route('/')
def home():
    return "ML Model API is running. Use /predict endpoint."

@app.route('/predict', methods=['POST'])
```

```
def predict():
    data = request.json

    features = np.array(data["data"]).reshape(1, -1)
    prediction = model.predict(features)

    return jsonify({
        "prediction": int(prediction[0])
    })
if __name__ == '__main__':
    app.run(debug=True, use_reloader=False)
```

OUTPUT:







ML Model API is running. Use /predict endpoint.

RESULT:

Thus, a Machine Learning model was successfully deployed using Flask, and Explainable AI using SHAP was applied to interpret model predictions by analyzing the contribution of each feature.